

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 06 · LECTURE NOTES

Consensus

Getting a group of machines to agree on one thing.

MODULE OUTCOMES

- Explain the replicated state machine approach.
- Explain Raft leader election and log replication.
- State the safety properties and why each is needed.
- Explain quorum intersection as the underlying mechanism.
- Explain membership change and the split-brain hazard.

1 Replicated state machines

AGREE ON THE LOG, NOT ON THE STATE

If every replica starts from the same initial state and applies the same deterministic operations in the same order, they necessarily arrive at the same state. The states are then equal without ever having been compared.

So replicating an arbitrary service reduces to agreeing on an ordered log of operations — one problem, solved once, rather than a bespoke replication scheme per service. This reduction is why consensus is a foundational building block rather than a curiosity.

Determinism is required, and it is easy to lose accidentally. Iterating a hash map in an unspecified order, reading the wall clock, drawing a random number, depending on pointer values, or floating-point summation order (CSCE 735 Module 13 §2) will all silently desynchronise replicas — and the divergence may not surface for hours.

This is also exactly how lockstep netcode works (Module 09 §3): every client runs the identical simulation on the identical inputs and therefore computes the identical world, with only the inputs transmitted. **The same idea in two very different settings**, with the same determinism hazard and the same consequence when it fails.

2 Raft

THREE ROLES: follower, candidate, leader. One leader per TERM;
a term is a logical clock -- an integer that only increases.

ELECTION

a follower hearing nothing for its RANDOMISED timeout
becomes a candidate, increments the term, votes for itself,
and requests votes
-> each node grants at most ONE vote per term
-> a majority of votes makes it leader
-> randomised timeouts make split votes rare

REPLICATION

clients send commands to the leader
leader appends to its log, sends AppendEntries
once a MAJORITY have stored it, the entry is COMMITTED
committed entries are applied to the state machine

ELECTION RESTRICTION -- the critical rule

a node votes for a candidate only if the candidate's log is
AT LEAST AS UP TO DATE as its own.

QUORUM INTERSECTION IS WHY ANY OF THIS WORKS

Any two majorities of the same set must share at least one member. Three out of five and three out of five cannot be disjoint; there are only five.

So a committed entry — which by definition is stored on a majority — is held by at least one member of any future election's voting majority.

Combined with the election restriction, a new leader therefore cannot be missing a committed entry. The member holding it would refuse to vote for a candidate whose log was behind, so no such candidate can assemble a majority.

That is the entire safety proof, in three sentences. It is the same argument underneath Paxos, underneath quorum reads and writes (Module 04 §1), and underneath every majority-based protocol in the subject.

Learn it once and it explains all of them — and conversely, whenever a protocol uses majorities, look for where the intersection argument is being made.

Property	Statement
Election safety	At most one leader is elected in any given term. Follows from one-vote-per-term plus majority.
Leader append-only	A leader never overwrites or deletes entries in its own log; it only appends.
Log matching	If two logs contain an entry with the same index and term, the logs are identical in all preceding entries. Maintained by the consistency check in AppendEntries.
Leader completeness	If an entry is committed in a term, it is present in the logs of all leaders of higher terms. This is the one the election restriction exists to establish, via quorum intersection.

Property	Statement
State machine safety	If a node applies an entry at a given index, no other node ever applies a different entry at that index. The property that actually matters to the application, and it follows from the others.

3 What goes wrong

SPLIT BRAIN, AND FENCING TOKENS

A leader that becomes partitioned may not immediately know it has lost leadership. It stops hearing from followers, but the absence of messages is exactly Module 01's ambiguity — it cannot distinguish 'everyone else is gone' from 'I am cut off'.

Majority writes stop automatically, since it cannot reach a quorum to commit anything. **But local reads continue, and any external side effect continues** — writing to shared storage, sending an email, charging a card, moving a robot.

And a process pause makes it worse. A long GC pause (CSCE 605 Module 12) or a VM migration (Module 03 §1) can leave a node that believes it holds a lease **resuming minutes after the lease expired and acting with authority it lost long ago**. This has caused real data corruption in real systems.

Fencing tokens are the fix. Every operation carries a monotonically increasing token obtained with the lease, and **the resource itself rejects any operation bearing a token lower than the highest it has already seen**. **The enforcement must be at the resource, not in the client** — a client that is paused cannot check anything, and a client that is confused cannot be trusted to check correctly.

MEMBERSHIP CHANGE IS WHERE IMPLEMENTATIONS BREAK

Changing the cluster from three nodes to five cannot be done atomically across all of them, so there is necessarily an interval during which different nodes disagree about who the members are.

If the old majority and the new majority do not intersect, two leaders can be elected simultaneously — one by a majority of the old configuration and one by a majority of the new. **The premise of the entire safety argument has been removed**, and both leaders believe themselves legitimate.

Raft's joint consensus introduces a transitional configuration in which a decision requires separate majorities of *both* the old and the new sets, which restores intersection throughout the change at the cost of real complexity.

Single-server changes — adding or removing exactly one node at a time — guarantee intersection automatically and are much simpler, which is what most implementations do. **Membership change is the most commonly broken part of real Raft implementations**, it is under-tested because it is rare in normal operation, and it is therefore exactly where a fault injector should be pointed.

4 Using consensus

- **Leader election for some other system.** By a wide margin the most common use: a consensus cluster decides which node of a separate, larger system is in charge.
- **Configuration and metadata.** Small data where correctness matters absolutely and throughput does not — which service is where, what the schema version is, which shard belongs to whom.

- **Distributed locks and leases**, with fencing tokens (§3). **A distributed lock without a fencing token is not safe**, regardless of how good the consensus underneath it is.
- **Replicated logs**, which then serve as the input to everything else — the log-as-source-of-truth pattern that Kafka and event sourcing are built on.
- **Not for bulk data**. Every write costs a round trip to a majority, so throughput is bounded by the slowest member of the fastest majority, and the data is stored on every node. **Consensus coordinates systems; it does not store their data**.
- **And use etcd, ZooKeeper, or Consul rather than writing your own** — while implementing Raft once anyway, because you cannot reason about what those systems promise without knowing how they promise it.

WHY CONSENSUS IS WRONG FOR A GAME

Consensus costs at least one round trip to a majority before anything is decided. That is intrinsic — it is what the quorum intersection argument requires.

At 60 Hz the frame budget is 16.7 ms, and a round trip to a geographically distributed majority is 50 to 150 ms. The arithmetic simply does not work, and no implementation quality fixes it.

So games do not reach agreement before acting. The client predicts the outcome immediately and is corrected afterwards (Module 10), and the server decides unilaterally rather than seeking consensus with anyone.

That is a strictly weaker guarantee, bought in exchange for meeting a hard deadline — and it is the right trade, because **an authoritative answer that arrives after the frame has been drawn is worth nothing at all**. This is the same shape of argument as CSCE 650's on latency and CSCE 605's on GC pauses: **for an interactive system, a worse answer on time beats a better answer late**.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Ongaro & Ousterhout — In Search of an Understandable Consensus Algorithm (free)	The Raft paper . Deliberately written to be implementable, and the interactive visualisation on the same site is excellent.
MIT 6.5840 — the Raft labs (free)	The best way to learn this is to implement it , and these labs have a test suite that finds the bugs you will have.
Lamport — Paxos Made Simple (free)	The other algorithm, from its author. Worth reading after Raft for the contrast in exposition.
Kleppmann — How to do distributed locking (free)	The fencing token argument of §3 , with the process-pause scenario worked through.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Raft leader election with randomised timeouts.
- 2 **Kill leaders repeatedly** and measure election latency as a distribution.
- 3 Implement log replication and the commit rule.
- 4 **Implement the election restriction**, then remove it and construct a history where a committed entry is lost.
- 5 Add persistence and verify a restarted node does not violate safety.
- 6 **Build a fault injector**: drop, delay, reorder, partition, crash, restart.
- 7 Run a linearizability checker over thousands of randomised histories with faults injected.
- 8 **Produce a split-brain scenario** with a paused leader, and then defeat it with fencing tokens.
- 9 Implement single-server membership change and test it under fault injection.
- 10 **Measure write throughput against cluster size** from 3 to 9 nodes and explain the curve.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does the replicated state machine approach reduce consensus to, and what does it require?
- 2 Name four ways determinism is accidentally lost.
- 3 Describe Raft election and the role of randomised timeouts.
- 4 State the election restriction and the quorum intersection argument.
- 5 Name Raft's five safety properties.
- 6 What is split brain, and why do fencing tokens have to be enforced at the resource?
- 7 Why is membership change dangerous, and what are the two approaches?
- 8 Name five uses of consensus and one non-use.
- 9 Why is consensus wrong for a 60 Hz game?